Exact verification guide and electronic supplement

Research notice. This is AI-generated research. The results, proofs, code and data have not yet been independently verified by human experts or formalized in Lean. Treat them as provisional; automated checks establish only their stated computational scope.

This guide accompanies *Rank-four Grothendieck rings through multiplicity 128*. It explains why the implementation checks the criteria proved there, including completeness of every finite search. It is not a second collection of case-by-case mathematical proofs.

1 Reproduction and scope

From the package root, install the dependencies recorded in `requirements.txt`, then regenerate and check the certificates:

```
python generate.py --workers 4
python verify.py --workers 4
```

The generator processes the pinned rank-four census through multiplicity 128. It applies the uniform criteria in the main paper and writes one proof record per input ring. The verifier reconstructs the tensors, checks each witness, and repeats every required bounded search. For shared signed certificates it first regenerates the complete global signed cover and reruns its universal arithmetic tests once. Each unsigned ring is then checked against every compatible surviving transform. The machine-generated report `verification/summary.json` records the realized, excluded and unresolved counts; an unresolved record is never interpreted as an exclusion. The criterion totals and complete ring-by-ring results are supplied in `verification/criterion_usage.json` and `verification/ledger.csv.gz`. The legacy tag `involution_propagation_42` is a certificate-schema name. Discovery and checking apply the general residual-involution criterion to complete low-induction branches, with no equality test against a particular multiplication tensor. The verifier reruns the full tensor audit against the pinned source.

The input consists of 889530 compact parameter records in `data/parameters_128.txt.gz`. The associated manifest fixes the repository commit and file digests. Reconstruction uses the two presentations of Appendix B; no approximate numerical fusion data are introduced. The source audit checks all fusion axioms and pairwise nonisomorphism under all six unit-fixing permutations. The explicit map in `data/through32_mapping.json.gz` preserves the labels at multiplicities at most 32. At multiplicities 33 through 128, labels follow the pinned parameter-file order within each multiplicity. Labels organize the records and are never premises of an exclusion.

The fusion-ring-census repository supplies the complete rank-four data and its coverage argument through multiplicity 128. The local parameter snapshot and manifest fix that source at commit `7f1b36e74df53a8a1cf8597db45d261e408d61b2`; the checker's input audit verifies tensor reconstruction, counts, isomorphism classes, and digests. Census completeness

is supplied by the pinned source repository; this package checks its input records rather than generating a second rank-four census. Runtime depends on the machine and is recorded separately from the mathematical certificates.

The supported environment is Python 3.10 or later with SymPy 1.14.0 and python-flint 0.8.0. FLINT accelerates exact integer and polynomial arithmetic. The full-tensor audit and signed-table accelerator need a C++17 compiler named `g++`. The signed-search Python reference is also supplied for cross-checking. No proprietary algebra system, numerical eigensolver or optimization solver is needed. The tested environment and source digests are recorded in the verification reports. Compiler output is temporary and is not a trusted stored executable.

Generation can be resumed from its arithmetic, direct-application and signed-search stages. The merged file `certificates/classification.jsonl.gz` is a compressed stream with a schema header followed by one record for each census ring. This format permits verification without keeping all tensors or proof trees in memory. Intermediate stage files are caches; the final verifier checks their mathematical assertions afresh.

The separate command `python verify_companion.py` checks the bounded arithmetic supporting the companion paper. Those results are not premises of the finite-classification verifier.

2 Files and mathematical interface

File or module	Role
`data/parameters_128.txt.gz`	All 889530 parameter records, with exact tensor reconstruction.
`data/census_128_manifest.json`	Pinned source, digests, counts and tensor-audit results.
`data/through32_mapping.json.gz`	Explicit isomorphisms preserving the labels through 32.
`data/models.json`	Fifteen full tensors with names and realization references.
`certificates/classification.jsonl.gz`	Streaming proof records for every input ring.
`software/census_data.py`	Pinned input, tensor reconstruction, labels and source checks.
`software/audit.py`	Tensor identity, axioms and exact multiplicities.
`software/applications.py`	Uniform applications and the full-tensor check of Larson's hypotheses.
`software/rings.py`	Ring recognizers, Casimirs and separating character coordinates.
`software/exact.py`	Rational polynomials, norm primes, field primes, exact roots and real signs.
`software/induction.py`	Exhaustive low restriction columns, Gram and twist-moment witnesses; quadratic-family certificates.
`software/signed.py`	Complete local signed searches and queries of the verified global cover.
`data/signed_universal_128.jsonl.gz`	Shared signed-orbit representatives surviving the universal criteria.
`software/global_signed.py`	Global-cover generation, exact arithmetic filters and complete replay.
`software/proofs.py`	Spherical reduction, global moments, signed parity and central-sign moments.
`software/dispatch.py`	Criterion dispatch, with no catalogue-index exclusion rules.

The final ledger is generated by verification, not trusted as an input. Identifiers organize results; they never prove an exclusion. A positive record requires a based-ring isomorphism to a full tensor in the model table. The existence of the named category is the cited mathematical input, not a new computer construction of associators.

The main paper proves the soundness implication in Proposition 8.1. Its standard inputs and proved uniform applications justify direct recognizers. Lemmas 2.3 and 2.4 justify the arithmetic witnesses. Proposition 3.3 justifies the low search; Proposition 2.5, Corollary 6.4, Proposition 6.2 and Corollary 7.2 justify the signed branches. Corollaries 7.3, 7.4 and 7.5 justify imposing their arithmetic, indicator and coefficient-domain constraints jointly. Proposition C.1 proves completeness of shared signed-cover queries. Every negative certificate ends at one of these criteria.

3 Exact character and induction calculations

The element $H = 1 + b_1 + kb_2 + k^2 b_3$, $1 \le k \le 13$, separates the four characters for at least one k. The checker verifies squarefreeness of its characteristic polynomial and all identities $N_i = q_i(N_H)$. For each irreducible factor p, the character values are the reductions of $q_i(t)$ in $\mathbb{Q}[t]/(p)$. Coefficients are reduced modulo p before being used as number-field elements. This last requirement matters when the component degree is smaller than four.

The positive component is identified using the strictly positive matrix N_H. Proposed roots of a codegree polynomial are supplied as rational coefficient vectors in this component field. The checker multiplies all linear factors and recovers the entire polynomial, including multiplicities; it does not accept a few sample roots. To reject a field inclusion, it checks a degree obstruction or the complete-splitting prime certificate from the main paper. A search failing to find such a prime is not treated as an obstruction.

For a low column v, the equation $\sum_i v_i d_i = D/f$ becomes a rational linear system in three unknowns after setting $v_0 = 1$. Row reduction identifies free coordinates. They are enumerated through the proven bounds

$$0 \le v_i \le \min\{B_{0i}, \lfloor \sqrt{B_{ii}} \rfloor\}.$$

Nonintegral or negative dependent coordinates are rejected. Two nonunit columns may be chosen explicitly; the third is determined by the required row sum and tested for membership in its complete column set. This is an exact enumeration, not an integer-programming heuristic.

For each remaining low part, the checker verifies a negative entry or principal minor of $B - LL^{\mathsf{T}}$, or a strictly positive excess in one of the two-moment inequalities. Every principal minor is calculated over $\mathbb{Q}$. Algebraic signs are determined by Horner bounds on a rational isolating interval, refined until zero is separated. A zero number-field element is recognized exactly; no tolerance replaces equality.

4 Completeness of the signed searches

4.1 Self-dual tables

Use the ten-coefficient matrices in Appendix B of the finite paper, or construct them from the symmetric products with independent tuple $(a, b, c, d, e, f, g, h, i, j)$. In this convention

the six associativity equations are

$$\begin{aligned} ad - b^2 + bg + ch - d^2 - e^2 + 1 &= 0, & ae - bc + bh + ci - de - ef &= 0, \\ be - cd + dh - eg + ei - fh &= 0, & af + bi - c^2 + cj - e^2 - f^2 + 1 &= 0, \\ bf - ce + di - eh + ej - fi &= 0, & df - e^2 + gi - h^2 + hj - i^2 + 1 &= 0. \end{aligned}$$

They are obtained by multiplying the three displayed regular matrices; conversely their vanishing gives all commuting-matrix identities and hence all multiplication identities. This is also tested against full matrix associativity in the small-input cross-checks.

Every signed coefficient runs through $-N, -N+2, \ldots, N$. A basis-sign change acts on the ten positions with binary masks

$$(1, 2, 4, 1, 7, 1, 2, 4, 2, 4).$$

An odd coefficient cannot vanish. Select linearly independent masks among odd positions and require those coefficients to be positive. Every sign orbit has a representative satisfying these conditions. Residual gauge symmetry is harmless: the search may contain several representatives of one orbit, but none is omitted.

The complete-domain C++ search exhausts a, b, c, d, e, f first. For $b, c \neq 0$, put

$$H = e(d + f - a) + bc, \quad G = b^2 + d^2 + e^2 - ad - 1, \quad J = c^2 + e^2 + f^2 - af - 1.$$

Set $A = e(b^2 - c^2) + bc(f - d)$ and $B = b^3e - b^2cd + b(d - f)H - ebG + ecH$. If $A \neq 0$, set $i = -B/A$; if $A = 0$, require $B = 0$ and exhaust i. Then determine $h = (H - ci)/b$, $g = (G - ch)/b$ and $j = (J - bi)/c$. For $b \neq 0, c = 0$, the fourth equation instead gives $i = J/b$; after solving for h, g, exhaust every bounded j. The independent Python reference still exhausts i before solving these three equations, providing a different implementation of this step. When $b = 0, c \neq 0$, determine

$$h = \frac{d^2 + e^2 - ad - 1}{c}, \quad i = \frac{e(d + f - a)}{c}, \quad j = \frac{c^2 + e^2 + f^2 - af - 1}{c},$$

and use $eg = h(d - f) + ei - cd$. For $e = 0$, require its right side to vanish and exhaust g; otherwise determine g by exact division.

Finally, when $b = c = 0$, first test

$$ad - d^2 - e^2 + 1 = 0, \quad e(a - d - f) = 0, \quad af - e^2 - f^2 + 1 = 0.$$

Exhaust h, i and use $eg = h(d-f)+ei$ and $ej = eh+i(f-d)$. For $e = 0$ both right sides must vanish and *both* g, j are exhausted. Every completed tuple is tested against all six equations and all original bounds and parities. This covers each possible denominator-zero case. Both implementations retain all zero branches; the extra linear elimination only accelerates the generic C++ branch.

4.2 Parity domains and Diophantine elimination

For the restricted self-dual search, Corollary 7.5 supplies at most eight disjoint congruence boxes. Each box is intersected with the same odd-coefficient positivity normalization described above. Duplicate and empty boxes are removed. The union over all compatible indicator signs is retained; no indicator normalization replaces the positivity gauge. Every omitted tuple is already excluded by adjoint parity.

Within a box each coordinate has an arithmetic-progression domain of step two or four. The implementation in `software/enumerate_signed_diophantine.cpp` exhausts b, c, d, e, f and, for $b, c \neq 0$, uses

$$Ca + Ai = -B_0,$$

with the polynomial coefficients from Appendix B. The identity holds for signed coefficients as well. Write the exact domains as $a = a_0 + s_a u$ and $i = i_0 + s_i v$, where s_a, s_i are two or four. The resulting linear Diophantine equation is solved by a greatest-common-divisor progression over the full bounded intervals for u, v. A zero coefficient is handled by solving for the other variable and exhausting the free one; if both coefficients vanish, the entire product of the domains is retained precisely when the constant term is zero. The remaining coordinates are then determined by the same division identities as in the complete-domain search.

If $b \neq 0, c = 0$, the fourth associativity equation instead gives $fa + bi = J_0$, treated by the same progression solver, with j left free. The branch $b = 0, c \neq 0$ uses the division formulas already displayed. If $b = c = 0$, then $e \neq 0$ forces $a = d + f$; for $e = 0$, the first equation forces $d \neq 0$ and $a = (d^2 - 1)/d$. Every resulting tuple is finally checked against all six original associativity equations and every original coefficient domain. These eliminations discard no admissible singular branch.

An additional exact filter reduces the six associativity equations modulo 16 in each parity box. It enumerates all permitted residue tuples, retaining the projection of their solutions onto the free coordinates (b, c, d, e, f, a, i). An absent outer projection or an absent pair (a, i) can be skipped before integer elimination. Every integer solution reduces to a solution of the modular equations, so this projection preserves completeness. Final integer equations are still checked in full.

4.3 Shared signed-cover queries

Appendix C of the main paper proves the completeness of a single signed cover shared among all self-dual input rings. The source `software/enumerate_global_signed.cpp` enumerates one representative of each orbit under the 48 nonunit basis permutations and basis-sign changes, with coefficient absolute values at most 128. The exact d-number and cyclotomicity tests exclude impossible signed orbits before any unsigned ring is queried. These exclusions are invariant under both actions.

The retained representatives and their Casimir determinants form `data/signed_universal_128.jsonl.gz`. Its manifest, `data/signed_universal_128_manifest.json`, records the bound, the number of global orbits, the universal rejection counts, the number of retained orbits, and source and data digests. The standalone regeneration command is

```
python build_signed_catalog.py --workers 4 --threads 4
```

The main generator creates this file when it is absent. A complete verification recompiles the enumerator, exhausts its global cover, reruns both universal arithmetic tests and compares the regenerated retained list and counts exactly with the supplied data. The resulting report is `verification/global_signed_catalog.json`; a digest check alone is not substituted for this mathematical replay.

Before expanding an orbit, the query applies a necessary residue signature test. In each of the nine adjoint-coefficient positions, record an even coefficient modulo four and replace an odd coefficient by one; for the remaining parameter e, record parity alone. Minimize this ten-entry signature over the six nonunit permutations. It is unchanged by basis-sign changes, and the adjoint-parity congruences force matching orbit signatures for a compatible signed and unsigned tensor. A mismatch therefore excludes the entire orbit. This coarse test retains all linked indicator-sign possibilities for the subsequent exact congruence check.

For a given unsigned ring R, the query expands every surviving orbit with a compatible signature into all distinct transforms. It retains precisely those satisfying R's coefficient bounds, coefficient parities, prescribed odd-coefficient positivity normalization, and adjoint-parity constraints. Required determinant-prime divisibilities are checked as well. Since the determinant is orbit-invariant, those tests may be performed before expansion. Every retained signed table receives the remaining character, spherical or central-moment certificate. The cover is common to all rings, but compatibility is checked against each entire unsigned tensor. No categorification status is inferred from an orbit label or an unsigned label.

4.4 Replay methods and counting conventions

The certificate's `method` field determines its replay source and the meaning of its count:

- `exhaustive_determinant_parity_v1` uses `software/enumerate_signed.cpp`. Its `total` counts all normalized associative tuples before the determinant and parity tests.
- `parity_domains_diophantine_v2` uses `software/enumerate_signed_diophantine.cpp`. Its `total` counts parity-admissible normalized associative tuples explicitly enumerated before the determinant tests. Tuples outside its boxes are excluded symbolically by the congruence theorem and are not counted.
- `global_signed_catalog_v3` queries the regenerated shared cover. The field `retained_tables` counts compatible signed tables left for the unsigned ring after the universal arithmetic, coefficient-domain, determinant and adjoint-parity filters.

The first two formats record the enumerator source digest, exact preliminary rejection counts and retained tuples. The third records both the global source and arithmetic-filter digests, the catalog digest, and the number of orbits surviving both the residue-signature and required-prime filters, in the field `eligible_orbits`. The verifier dispatches by the recorded method and regenerates its complete candidate set. The Python reference enumerates full local domains and can check both the restricted union and the shared cover on bounded examples independently.

The statistics distinguish three quantities: global signed orbits; global orbits retained by the universal criteria; and per-ring candidates explicitly processed by each method. Global counts occur once in `verification/global_signed_catalog.json`; per-method counts occur in `verification/criterion_usage.json`. Their sum is a workload count with the stated filtering conventions, not a count of all unrestricted associative signed tables. Overlap between candidates compatible with different unsigned rings is expected. Regenerating witnesses may select a different supported method and hence change these workload counts without changing the classification. Replaying the supplied certificate stream reproduces its recorded allocation.

4.5 Non-self-dual tables

Normalize duality to $[0, 1, 3, 2]$. The six-parameter matrices are

$$T_X = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & a & b & b \\ 0 & b & c & d \\ 0 & b & d & c \end{pmatrix}, \quad T_Y = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & b & c & d \\ 0 & d & e & f \\ 1 & c & e & e \end{pmatrix}, \quad T_Z = T_Y^{\mathsf{T}}.$$

The four equations are

$$ac - b^2 + 2be - c^2 - d^2 + 1 = 0, \qquad ad - b^2 + be + bf - 2cd = 0,$$
$$b(c - d) + d(e - f) = 0, \qquad -c^2 + d^2 - e^2 + f^2 - 1 = 0.$$

The accelerated search exhausts c, d, e. The last equation gives $f^2 = e^2 + c^2 - d^2 + 1$; an exact integer-square test retains both roots allowed by the original bound and parity, counting zero once. If $c \neq d$, the third equation determines $b = d(f-e)/(c-d)$, subject to exact divisibility and its original bound and parity. If $c = d$, require $d(e-f) = 0$ and exhaust all allowed b. The first equation determines a when $c \neq 0$; the second does so when $c = 0 \neq d$. If both vanish, exhaust all allowed a. Every completed tuple is retested against all four equations and all original bounds and parities. The independent Python implementation exhausts b, c, d, e, f before solving for a, giving a separate check of the elimination. The two sign actions change (a, c, d) and (b, e, f) independently; taking the lexicographically least image retains exactly one representative of each such orbit.

5 Coverage, stress tests and remaining trust

The accepted run covers the entire 889530-ring input census. The numbers of signed records and explicitly enumerated candidates are generated in `verification/summary.json`, with the counting convention specified there and a breakdown by method in `verification/criterion_usage.json`. The verifier also re-enumerates every low-induction and central-sign branch. The latter checks all allowed second indicators and all sign-object twist patterns at every real embedding, with the same rational signs under Galois conjugation. Indicator assignments are first intersected with all adjoint-parity congruences. Spherical global-moment branches use the parity-refined upper bound when an odd adjoint coefficient forces an indicator to one. The d-number and cyclotomicity conditions of Corollary 7.3 are checked once globally for shared signed-cover queries, and within the signed branch for local searches. These steps combine the necessary conditions before more expensive character searches.

The command `python test_applications.py` checks the uniform applications under basis permutations. Unspecified exclusion-classification tags and deliberately altered application witnesses are rejected. Larson's theorem has its own explicit certificate kind with full hypothesis checks. The extension tests also reject false parity, global-moment and literature witnesses and compare the Bareiss Casimir determinant with symbolic determinants.

The regression suite compares the Python and C++ signed searches on all signed inputs through multiplicity six, compares optimized elimination against direct small bounded enumeration, and deliberately corrupts certificates. Omitting a signed or induction branch, altering a codegree root, supplying a false divisibility witness or assigning a false positive model must be rejected. The results are stored in `verification/tests.json`.

The arithmetic relies on Python, SymPy and FLINT exact polynomial and number-field operations, and either the supplied integer C++ enumeration or the Python reference. The C++ accelerator guards its coefficient range so all signed arithmetic fits in its integer type: the C++ enumerators enforce $M \leq 1000$. The census uses signed 128-bit intermediates for modular products and 64-bit integers elsewhere. Larger signed searches require the Python backend. Casimir determinants use fraction-free Bareiss elimination with arbitrary-precision Python integers; every exact division is checked. Its pivots are nonzero because $B = \sum_i N_i N_i^{\mathsf{T}}$ is positive definite (the unit matrix is one summand). These are ordinary reproducible computer-assisted proofs, not formally certified machine code or proof-assistant formalizations. The categorical lemmas and published realizations remain mathematical inputs subject to normal independent review. No previously stored classification status is among those inputs.